

SDEV2171

Lesson 16: Forms and Saved State

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, you should be able to:

- Build controlled text inputs in React Native
- Keep a form’s draft in local state, and decide when it becomes shared state
- Validate a draft by deriving its errors, and show them at the right moment
- Save a committed result into a context, and show it on other screens
- Explain when a copy of state is a bug and when it’s the point
- Refactor repeated form markup into a reusable field component

WE ARE ESSENTIAL TO ALBERTA | NAIT

Today

Time	Block
0:00	Run the backend and the starter app
0:05	Draft state and saved state
0:15	The session form, with controlled inputs
0:30	Validating the draft
0:45	Break
0:55	Adding the session to the context
1:03	Saving the session
1:15	Refactoring into a field component
1:23	How production apps do this
1:30	Challenges and checkpoint

WE ARE ESSENTIAL TO ALBERTA | NAIT

Run the Backend and the Starter App

The backend is the same one as lessons 13 and 15. In its own terminal, run `node server.js` from the `backend` folder, and leave it running.

In a second terminal, from the `starter-app` folder:

```
npm install
npm start
```

Press `w` for the browser, or open your development build. The config and dependencies haven't changed since lesson 13, so your development build still works.

The starter is the finished lesson 15 app. `StudyPlanContext` holds the plan, and Home, Study Steps and Plan all read it. Add three steps and check that all three screens agree before you change anything.

WE ARE ESSENTIAL TO ALBERTA | NAIT

What We’re Building

A plan is useful for one study session. Today the user names the session and saves it:

Screen	What changes
Plan	a Save as session button, and a card showing the saved session
Save Session	a new screen with a title, an optional goal, and a Save button
Home	a line showing the saved session’s title

The form only saves when the title isn’t blank and the plan has at least one step. Saving stores the title, the goal, and the steps in the plan at that moment.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Draft State and Saved State

A form has two kinds of state, and they live in different places.

	Draft	Saved session
What it is	what the user is typing right now	what the user chose to keep
Changes	on every keystroke	once, when the user presses Save
Who needs it	the form screen only	the form, Plan, and Home
Where it lives	`useState` in the form screen	the context
If the user leaves	it’s thrown away	it stays

The Save button is the line between them. Pressing it is the user saying "this is what I meant."

WE ARE ESSENTIAL TO ALBERTA | NAIT

A Draft Is a Copy on Purpose

In lesson 15, the Plan screen kept its own copy of the plan, and that was the bug: two copies of something that was supposed to be one.

A form draft is also a copy, and that’s the point. The user should be able to type, change their mind, and leave without touching the saved data. When they press Save, the draft replaces the saved version in one step.

The difference is intent. Duplicated state is two owners of data that should always match. A draft is data that isn’t supposed to match yet.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why Not Type Straight Into the Context

You could store the title in the context and update it on every keystroke. Three things go wrong:

1. Every component that reads the context re-renders on every keystroke, including screens the user can’t see
2. Home and Plan show a half-typed title, like `Week 3 la`, before the user has decided anything
3. There’s no way to cancel. The old title is gone as soon as the user types the first letter

Keep each keystroke in the form. Put only the result in the context.

WE ARE ESSENTIAL TO ALBERTA | NAIT

How to Build and Commit a Change

Build every change in this order:

1. Write the shape first, with hard-coded values
2. Connect everything that uses it
3. Check that the app runs and shows the hard-coded values
4. Replace the hard-coded values with the real logic
5. Test the whole flow
6. Commit

For a form, the shape is the screen and its state. For saving, it’s the new values in the context’s contract.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Building the Session Form

The app runs after each of these changes:

Change	What you check after it
Add the form screen with controlled inputs	typing works, and Save logs the draft
Validate the draft	errors appear after the first Save, and clear as you fix them
Add the saved session to the context, hard-coded	Plan and Home show a test session
Save the session and leave the form	the saved session appears on Plan and Home
Extract a field component	nothing visible changes

WE ARE ESSENTIAL TO ALBERTA | NAIT

Add the Form Route

Create `src/app/session-form.js`. Register it in `src/app/_layout.tsx`, after the `plan` screen:

```
<Stack.Screen name="session-form" options={{ title: 'Save Session' }} />
```

On the Plan screen, add a button above Go back that opens it:

```
<Pressable style={styles.button} onPress={() => router.push('/session-form')}>
  <Text style={styles.buttonLabel}>Save as session</Text>
</Pressable>
```

The form opens from Plan because a session is a snapshot of the plan. The user looks at their plan, then saves it.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Controlled Inputs in React Native

A controlled input shows a value from state, and sends every change back to state:

```
const [title, setTitle] = useState('');

<TextInput value={title} onChangeText={setTitle} />
```

- `onChangeText` gives you the new text as a string. `onChange` gives you an event, and the text is in `event.nativeEvent.text`. Use `onChangeText`
- A `TextInput` value is always a string, even with `keyboardType="numeric"`. Convert it yourself when you need a number
- The input shows `title`, not what was typed. Remove `onChangeText`, and typing does nothing

Controlled inputs make the draft a normal value you can validate, trim, and save.

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Session Form

The README has the full file, with the styles. `maxLength={40}` stops a 41st character, so length never needs an error.

```
export default function SessionFormScreen() {
  const { selectedSteps } = useStudyPlan();
  const [title, setTitle] = useState('');
  const [goal, setGoal] = useState('');
  function handleSave() {
    // TODO: validate, then save to the context
    console.log('draft', { title, goal });
  }
  return (
    <ScrollView contentContainerStyle={styles.screen} keyboardShouldPersistTaps="handled">
      <Text style={styles.intro}>Steps in this session: {selectedSteps.length}</Text>
      {/* Title: a TextInput with value={title}, onChangeText={setTitle}, maxLength={40} */}
      {/* Goal: the same with goal and setGoal, plus multiline */}
      <Pressable style={styles.button} onPress={handleSave}>
        <Text style={styles.buttonLabel}>Save session</Text>
      </Pressable>
    </ScrollView>
  );
}
```

WE ARE ESSENTIAL TO ALBERTA
NAIT

Trim When You Save, Not While Typing

`onChangeText={(text) => setTitle(text.trim())}` looks tidy, but it breaks typing. Type `Week 3` : after `Week` the trailing space is removed before the `3` arrives, and the title becomes `Week3` .

Let the draft hold exactly what the user typed. Trim once, when you validate and when you save:

```
saveSession({ title: title.trim(), goal: goal.trim() });
```

Type `Week 3` and press Save. The console logs the spaces. They’re part of the draft, and they’ll be removed on save.

WE ARE ESSENTIAL TO ALBERTA | NAIT

What Counts as a Valid Session

Rule	Message	Shown
The title isn’t blank after trimming	Give the session a title.	under the Title field
The plan has at least one step	Add at least one step to your plan before saving.	above the Save button

The first rule is a **field error**: it belongs to one input, so it appears next to that input.

The second is a **form error**: no field can fix it. The user has to go back and add steps. It appears near the button, where the user is looking when it happens.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Errors Are Derived From the Draft

Write validation as a function that takes the draft and returns one message per problem:

```
function validateSession({ title, steps }) {
  const errors = {};
  if (title.trim() === '') {
    errors.title = 'Give the session a title.';
  }
  if (steps.length === 0) {
    errors.steps = 'Add at least one step to your plan before saving.';
  }
  return errors;
}
```

Call it on every render, and don’t store the result in state:

```
const errors = validateSession({ title, steps: selectedSteps });
const isValid = Object.keys(errors).length === 0;
```

An `errorMessage` state goes stale: it keeps showing after the user fixes the title, until they press Save again. Derived errors are recalculated from the draft, so they’re always current.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Show Errors After the First Save Attempt

An empty form is invalid, but red text before the user has typed anything tells them nothing. Keep one flag:

```
const [hasTriedToSave, setHasTriedToSave] = useState(false);

const titleError = hasTriedToSave ? errors.title : undefined;
const stepsError = hasTriedToSave ? errors.steps : undefined;

function handleSave() {
  setHasTriedToSave(true);
  if (!isValid) return;
  console.log('valid draft', { title: title.trim(), goal: goal.trim() });
}
```

`hasTriedToSave` is state, because it records something that happened. The errors aren't, because they're computed from the draft.

After the first attempt, errors disappear the moment the user fixes them, and they never appear while the user is still filling in the form for the first time.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Showing the Errors

Under the title input, and in the input's style:

```
style={[styles.input, titleError && styles.inputError]}

{titleError && <Text style={styles.errorText}>{titleError}</Text>}
```

Above the Save button:

```
{stepsError && <Text style={styles.formError}>{stepsError}</Text>}
```

`titleError` is either a message or `undefined`, so `&&` is safe here. Be careful with `&&` on a value that can be an empty string. `{goal && <Text>...</Text>}` renders `''` when the goal is empty, and React Native crashes with "Text strings must be rendered within a `<Text>` component." Compare explicitly: `{goal !== '' && ...}`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Add the Session to the Context

Extend the contract with hard-coded values first, in the provider:

```
// TODO: replace these hard-coded values with real state
const savedSession = null;
function saveSession() {}

const value = { selectedSteps, toggleStep, removeStep, savedSession, saveSession };
```

A saved session will look like `{ title, goal, steps }`, or `null` before the first save.

Then connect the screens that show it. Change `null` to `{ title: 'Test session', goal: '', steps: [] }`, save, and check both screens. Put it back to `null` before you commit.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Show the Saved Session

On the Plan screen, read `savedSession`, and add this above the count:

```
{savedSession ? (
  <View style={styles.card}>
    <Text style={styles.cardLabel}>Saved session</Text>
    <Text style={styles.cardTitle}>{savedSession.title}</Text>
    {savedSession.goal !== '' && <Text style={styles.cardBody}>{savedSession.goal}</Text>}
    <Text style={styles.cardBody}>Steps saved: {savedSession.steps.length}</Text>
  </View>
) : (
  <Text style={styles.empty}>No saved session yet.</Text>
)}
```

On Home, below the plan count:

```
{savedSession && <Text style={styles.body}>Saved session: {savedSession.title}</Text>}
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Save, Then Leave the Form

In the provider, replace the hard-coded values:

```
const [savedSession, setSavedSession] = useState(null);

function saveSession({ title, goal }) {
  // The session keeps the plan as it was when it was saved.
  setSavedSession({ title, goal, steps: selectedSteps });
}
```

The provider owns the plan, so it attaches the steps. The form only sends what the user typed.

In the form, replace the `console.log`:

```
saveSession({ title: title.trim(), goal: goal.trim() });
router.back();
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Go Back Instead of Pushing

After saving, the user should land on the Plan screen. `router.push('/plan')` gets them there, but it adds a second Plan screen on top of the form:

Stack after saving with	Pressing back from Plan shows
`router.push('/plan')`	the form, still filled in
`router.back()`	Study Steps

The form opened from Plan, so going back returns to the same Plan screen, which already shows the new session. Leave a form by removing it from the stack, so the user can’t go back into a draft they’ve already saved.

WE ARE ESSENTIAL TO ALBERTA | NAIT

A Snapshot That Shouldn’t Update

Save a session with three steps, then remove a step on the Plan screen. The plan says 2, and the session card still says `Steps saved: 3`.

That’s correct. The session is a record of the plan when the user saved it, so it shouldn’t follow later changes. In lesson 15 a copy that didn’t update was a bug, because both copies were meant to be the same list. This copy is meant to stay as it was.

The snapshot costs nothing. `steps: selectedSteps` keeps a reference to the current array. Every plan update creates a new array instead of changing the old one, so the array the session points to never changes.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Test the Whole Flow

Do this	Expected
Open the form with an empty plan, press Save	both errors
Type a title	the title error disappears as you type
Add three steps, open the form, type `Week 3 prep`, press Save	back on Plan: a card with `Week 3 prep` and `Steps saved: 3`
Remove a step on Plan	the plan shows 2, the card still shows 3
Go back to Home	`Saved session: Week 3 prep`
Open the form again	the fields are empty

The last row is a new draft. The form unmounted when you left it, so its state started over.

WE ARE ESSENTIAL TO ALBERTA | NAIT

One Component per Field

Each field repeats the same markup: a label, an input, an error, and the error border. Move it into `src/components/FormField.js`:

```
export function FormField({ label, error, style, ...inputProps }) {
  return (
    <View style={styles.field}>
      <Text style={styles.label}>{label}</Text>
      <TextInput {...inputProps} style={[styles.input, style, error && styles.inputError]} />
      {error && <Text style={styles.errorText}>{error}</Text>}
    </View>
  );
}
```

`...inputProps` collects every other prop, and passes it straight to the `TextInput`. `FormField` handles the layout and the error, and the screen still controls the input.

Move the `field`, `label`, `input`, `inputError` and `errorText` styles with it.

WE ARE ESSENTIAL TO ALBERTA | NAIT

The Form With FormField

```
<FormField
  label="Title"
  value={title}
  onChangeText={setTitle}
  placeholder="Week 3 lab prep"
  maxLength={40}
  error={titleError}
/>

<FormField
  label="Goal (optional)"
  value={goal}
  onChangeText={setGoal}
  placeholder="What should this session get done?"
  multiline
  style={styles.multiline}
/>
```

Run the flow from the test slide again. Nothing should look or behave differently. That’s how you know a refactor is done.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Form Libraries: React Hook Form

Large forms get repetitive: state per field, touched flags, errors, submit handling. Most production apps use a library, and React Hook Form is a common choice in React Native.

```
const { control, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(sessionSchema),
  defaultValues: { title: '', goal: '' },
});

<Controller
  control={control}
  name="title"
  render={({ field: { value, onChange, onBlur } }) => (
    <FormField label="Title" value={value} onChangeText={onChange} onBlur={onBlur}
      error={errors.title?.message} />
  )}
/>

<Pressable onPress={handleSubmit(onValid)}>
```

React Native has no HTML form elements, so each input goes through `Controller`. `handleSubmit` validates, and calls `onValid` only when the values pass.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Schema Validation With Zod

A schema describes valid data once. Zod is the usual choice with React Hook Form, through `zodResolver`:

```
const sessionSchema = z.object({
  title: z.string().trim().min(1, 'Give the session a title.'),
  goal: z.string().trim(),
});
```

- `safeParse(draft)` returns either `{ success: true, data }` or `{ success: false, error }`. `data` is already trimmed
- The same schema can run on a Node server, so the app and the API enforce the same rules
- A server still has to validate. Anyone can send a request without using your app

Today's `validateSession` is what a schema replaces. The last challenge swaps one in.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Forms on a Phone

The browser hides problems a phone shows:

- **Tapping Save with the keyboard open.** By default, a `ScrollView` uses the first tap to close the keyboard, and the button doesn't get it. `keyboardShouldPersistTaps="handled"` lets the button take the tap
- **The keyboard covers the inputs.** `KeyboardAvoidingView` moves the content up. The Expo docs say to set its `behavior` on both platforms, because Expo apps on Android draw under the keyboard
- **Long forms.** `react-native-keyboard-controller` adds a `KeyboardAwareScrollView` that scrolls to the focused input. Install it with `npx expo install`. It needs `react-native-reanimated`, which the app already has
- **The right keyboard.** `keyboardType`, `autoCapitalize`, `autoComplete` and `returnKeyType` change which keyboard appears and what its Enter key does

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenges

Pick one of these challenges to finish after class. Each slide explains the problem briefly. Look up the term it names, read until you understand it, then implement a solution.

Each challenge gives you:

- what to do in the app to see the problem
- what your solution has to do
- a hint, and a starter snippet

Build it on top of your finished form, and commit it on its own, following the build order. The README has a full solution for each one in a collapsed section. Try it yourself before you look.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Edit the Saved Session

Every time the form opens, it starts empty, even when a session is already saved. Look up **initializing state from props** in the React docs.

Trigger it: save a session, then press Save as session again. Both fields are empty.

Your solution: when a session is saved, the form opens with its title and goal filled in, and the button says Update session. On the Plan screen, the button says Edit session. Saving replaces the session, with the current plan's steps.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Edit the Saved Session: Hint

Hint: `useState` only uses its starting value on the first render. In lesson 15, that made the Plan screen's copy go stale. For a draft, it's exactly right: the draft starts as a copy of the saved session, then changes on its own until the user saves.

```
const { selectedSteps, savedSession, saveSession } = useStudyPlan();

// TODO: start title and goal from savedSession, or '' when there isn't one
const [title, setTitle] = useState(/* TODO */);
const [goal, setGoal] = useState(/* TODO */);

// TODO: label the button 'Update session' when savedSession exists
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Move Between Fields

On a phone, the user expects the keyboard's Enter key on the title to move to the goal. Right now it closes the keyboard. Look up **focusing a TextInput with a ref**.

Trigger it: click into Title, type, and press Enter. Nothing moves to the goal field.

Your solution: pressing Enter in Title puts the cursor in Goal. On a phone, the Enter key on the title's keyboard says Next.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Move Between Fields: Hint

Hint: a ref gives you the `TextInput` itself, which has a `focus()` method. In React 19, `ref` is a normal prop, so `FormField` already passes it to the `TextInput` through `...inputProps`.

```
const goalRef = useRef(null);

<FormField
  label="Title"
  returnKeyType="next"
  submitBehavior="submit"
  onSubmitEditing={() => {/* TODO: focus the goal field */}}
  // ...the props it already has
/>

<FormField label="Goal (optional)" ref={goalRef} /* ...the props it already has */ />
```

`submitBehavior="submit"` keeps the keyboard open while focus moves, instead of closing it and opening it again.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Validate When a Field Loses Focus

Errors only appear after Save. A user who skips the title and moves on finds out at the end. Look up **touched fields** in form validation.

Trigger it: click into Title, then click into Goal without typing. No error appears.

Your solution: when the user leaves the title empty and moves to another field, its error appears straight away. It still disappears as soon as they type a title.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Validate on Focus Loss: Hint

Hint: a field is "touched" once it has lost focus. Track touched fields in one object, and show a field's error once it's touched or the user has tried to save.

```
const [touched, setTouched] = useState({});

const titleError = /* TODO: hasTriedToSave or touched.title */ ? errors.title : undefined;

<FormField
  label="Title"
  onBlur={() => {/* TODO: set touched.title to true, keeping the others */}}
  // ...the props it already has
/>
```

`onBlur` runs when the input loses focus.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Challenge: Validate With a Schema

`validateSession` is hand-written. A server that accepts sessions would need the same rules, written again. Look up **Zod safeParse**.

Trigger it: read `validateSession`, and imagine adding five more rules to it.

Your solution: replace the body of `validateSession` with a Zod schema. It must return exactly what it returns now: one message per field, and `{}` when the draft is valid. Nothing else in the screen changes.

Install it with `npm install zod`. It's plain JavaScript, so it needs no rebuild.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Validate With a Schema: Hint

Hint: `safeParse` never throws. When it fails, `result.error.issues` is a list, and each issue has a `path` (like `['title']`) and a `message`.

```
const sessionSchema = z.object({
  title: z.string().trim().min(1, 'Give the session a title.'),
  steps: z.array(z.unknown()).min(1, 'Add at least one step to your plan before saving.'),
});

function validateSession(draft) {
  const result = sessionSchema.safeParse(draft);
  if (result.success) return {};
  // TODO: build { title?: message, steps?: message } from result.error.issues
  //       keep the first message for each field
}
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Checkpoint

- Which form values are draft state, and where do they live?
- Why are the errors calculated on every render instead of stored in state?
- What does `hasTriedToSave` change, and why is it state when the errors aren't?
- Why does the form call `router.back()` instead of `router.push('/plan')`?
- The session card says 3 steps and the plan says 2. Is that a bug? How is it different from lesson 15?

WE ARE ESSENTIAL TO ALBERTA | NAIT

Before You Leave

1. Check the whole flow: both errors, a valid save, the card on Plan, and the title on Home
2. Check that no `console.log` from the build is left in the form
3. Commit and push your work
4. Record your status: **resolved**, **partial**, or **blocked**
5. Pick a challenge, and do the exercise at the end of the lesson README

WE ARE ESSENTIAL TO ALBERTA | NAIT